现代卷积神经网络

虽然深度神经网络的概念非常简单——将神经网络堆叠在一起。但由于不同的网络架构和超参数选择，这些神经网络的性能会发生很大变化。本章介绍的神经网络是将人类直觉和相关数学见解结合后，经过大量研究试错后的结晶。

深度卷积神经网络（AlexNet）

意义：证明了深层卷积神经网络能有效完成真实大规模视觉任务。

AlexNet 和 LeNet 的构造区别：

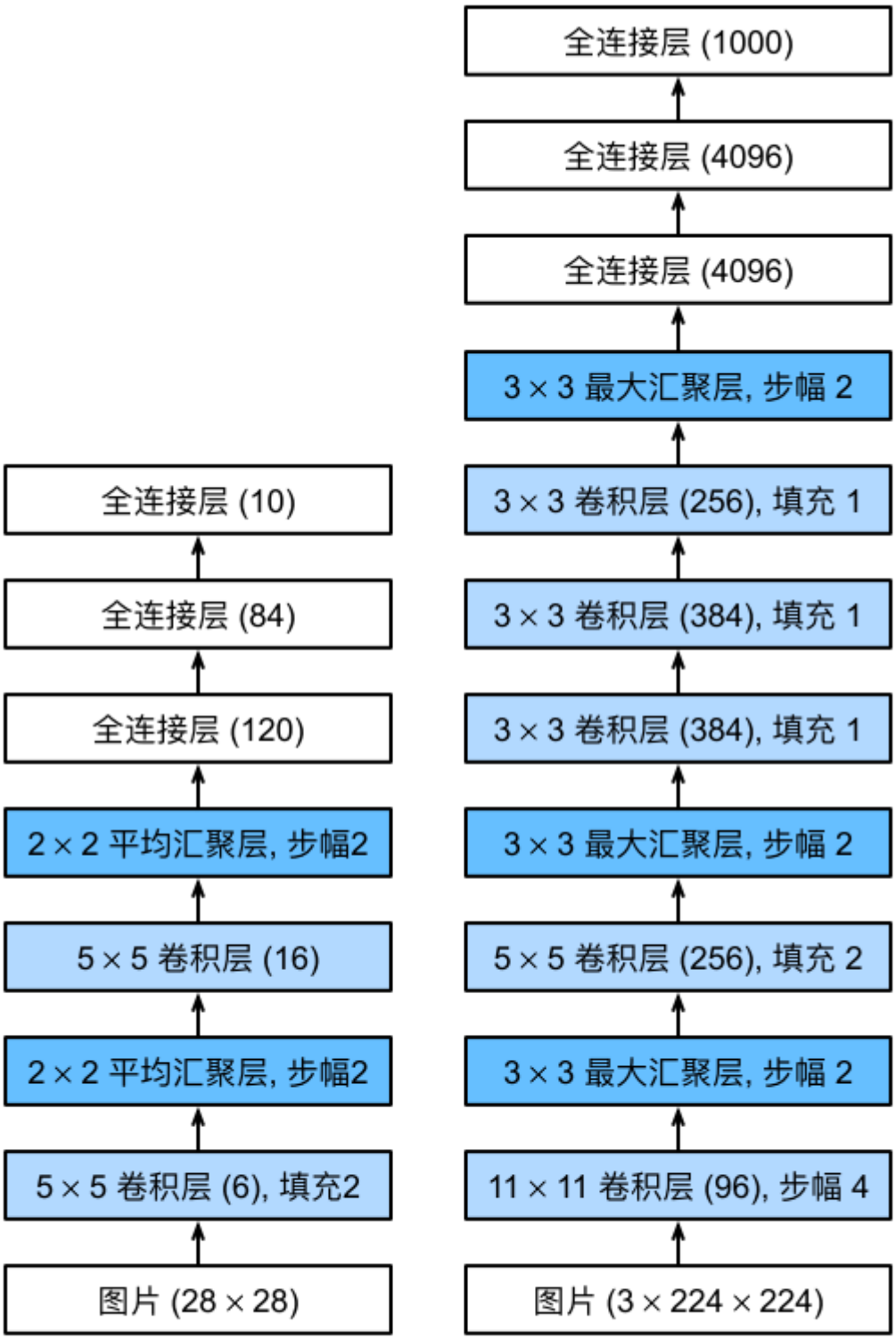


图7.1.2: 从LeNet（左）到AlexNet（右）

主要区别：

1. **深度更深** -> 能够学习更加整体的特征

（浅层看局部特征，中层看局部特征构成的部件，深层看部件构成的整体）

（理解：浅层的元素的感受野小，只能看到非常局部的细节特征；深层元素的感受野很大，甚至覆盖整个输入，能够综合所有的小特征，也就是看整体）

通道数更多 -> 能够同时表征更多种类的不同特征

（理解：输出通道数就是输出多少张处理后的图像/总结出的数字，不同通道由于卷积核的差异学习到不同的特征，最后表现出的特征数量也就多）

2. 采用 ReLU 函数作为激活函数而非 sigmoid

（原因在前面几章介绍过，防止多层梯度相乘消失）

3. AlexNet 相较 LeNet 多使用了暂退法来训练，控制模型复杂度。

4. AlexNet 在训练时增加了大量的图像增强数据，如翻转、裁切和变色。这使得模型更健壮，更大的样本量有效地减少了过拟合。

使用块的网络（VGG）

VGG 网络由**两大部分**组成：前面的**卷积层** + **汇聚层**；后面的**全连接层**

VGG 块：将前面的重复单元：**卷积层和汇聚层打包成一个 VGG 块**，方便复用

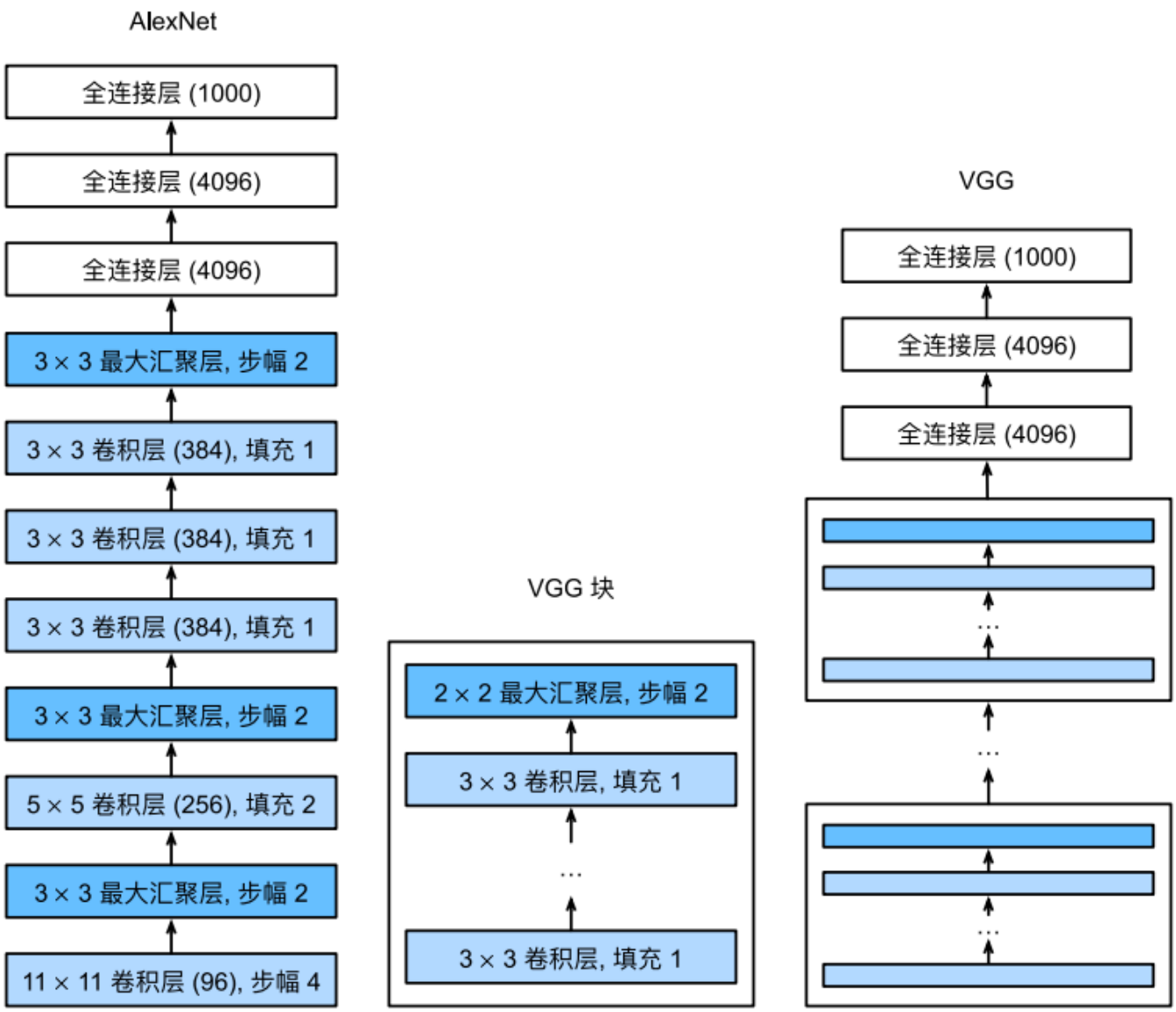


图7.2.1: 从AlexNet到VGG，它们本质上都是块设计。

网络中的网络（NiN）

构成：由一个**大卷积层和两个** 1×1 **卷积层**（相当于一个小型网络）构成，层和层之间有激活函数提供非线性能力

NiN 模型：在每个空间位置上，通过多层 1×1 卷积层和激活函数，来整合多通道的信息；**完全抛弃了之前最后的全连接层，而是改用全局平均汇聚代替**

传统网络：卷积提特征 → 展平 → 全连接层分类

NiN：卷积提特征 → **每个通道对应某类语义** → 全局平均 → 分类

（传统网络前半部分只负责提取特征，由特征形成判断是后半部分的全连接层的工作；NiN 则是让卷积部分本身就学会和类别相关的特征，然后直接根据卷积部分的结果来分类）

好处：减小全连接层带来的大参数；卷积特征与分类结果之间的对应更加清楚。同时可以减小过拟合

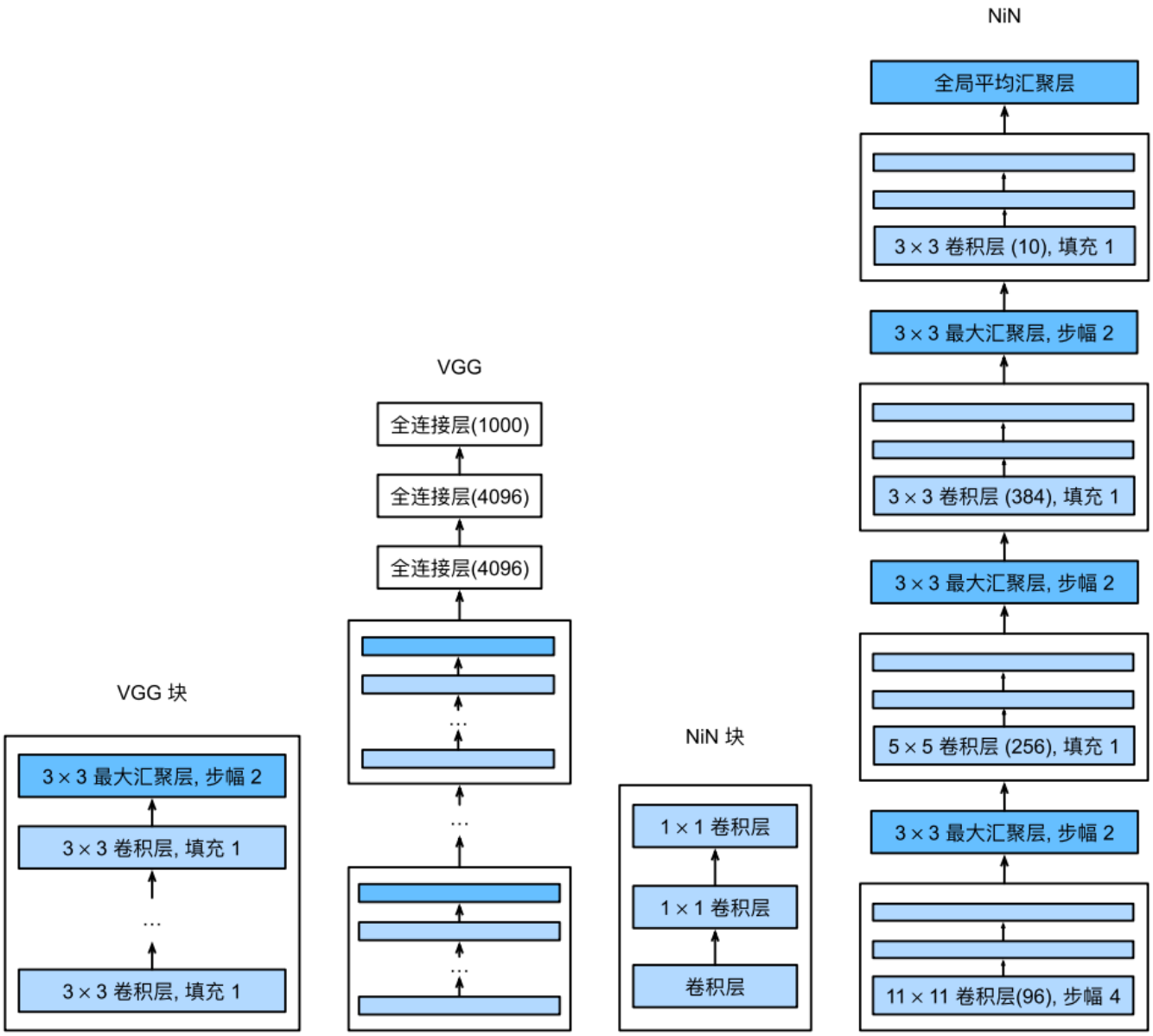


图7.3.1: 对比 VGG 和 NiN 及它们的块之间主要架构差异。

为什么能够减轻过拟合？其实和参数量一脉相承：全连接层参数多、自由度大，容易记住训练集细节；全局平均汇聚参数少、结构更受限制，迫使卷积层学到更通用的类别特征，所以可以减轻过拟合。

含并行连结的网络（GoogLeNet）- Inception 块

之前的网络结构都是线性的，而 inception 块引入了**并行**的模式

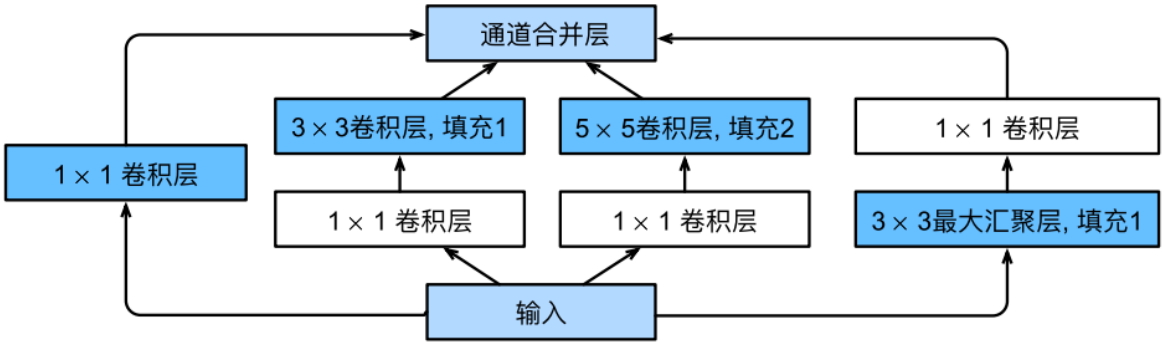


图7.4.1: Inception块的架构。

具体执行：相当于四个单线的处理同时进行，最后在通道合并层，每一条线的通道之间互不影响，只是**所有的通道合并在一起**

注意通道数和每个通道内数据的长和宽不是一个概念。四条线可以得到**不同数量的通道**，但是必须保证**每个通道内数据的长和宽分别相等。**

Eg：四条线分别得到了 $64, 128, 32, 32$ 通道，则最后输出的是 $64 + 128 + 32 + 32 = 256$ 通道

但是每条线得到的单个通道内的矩阵都必须是 128×64 的，否则无法拼接

好处：相当于同时用不同大小的卷积核来提取不同范围，不同尺度内的特征

"用不同倍数的放大镜来看图像，提取出不同大小的特征，从局部到整体"

批量规范化（BN）

前面这些是对模型构成本身的调整，模块化和创新，而为了训练的稳定性以及实现更深的训练层数，还需要在训练时进行批量规范化。

为什么需要？训练中，前一层提供给后一层的数据每次都会发生较大变化，使得后一层的学习变得困难（因为要一直适应前一层的大变化）。

批量规范化就是要解决这个问题：让每一层接收到的数据保持在比较稳定的范围内。

具体做法：分成两步，首先是标准化，然后是尺度和偏移训练：

1. **标准化**：和随机变量的标准化是一样的，假设处理前的一组数为 x，则处理后为

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

其中：

- μ：当前小批量的均值
- σ^2：当前小批量的方差
- ϵ：很小的数，防止除以 0

这样就标准化了这组数据，使得其均值接近 0，方差接近 1（让数据分布更稳定）

2. **尺度和偏移训练**：如果强制每一层的输出都是均值 0 方差 1，可能会限制网络的表达能力。因此，需要对标准化后的数据增加两个可以学习的参数 γ 和 β，处理后数据为 y：

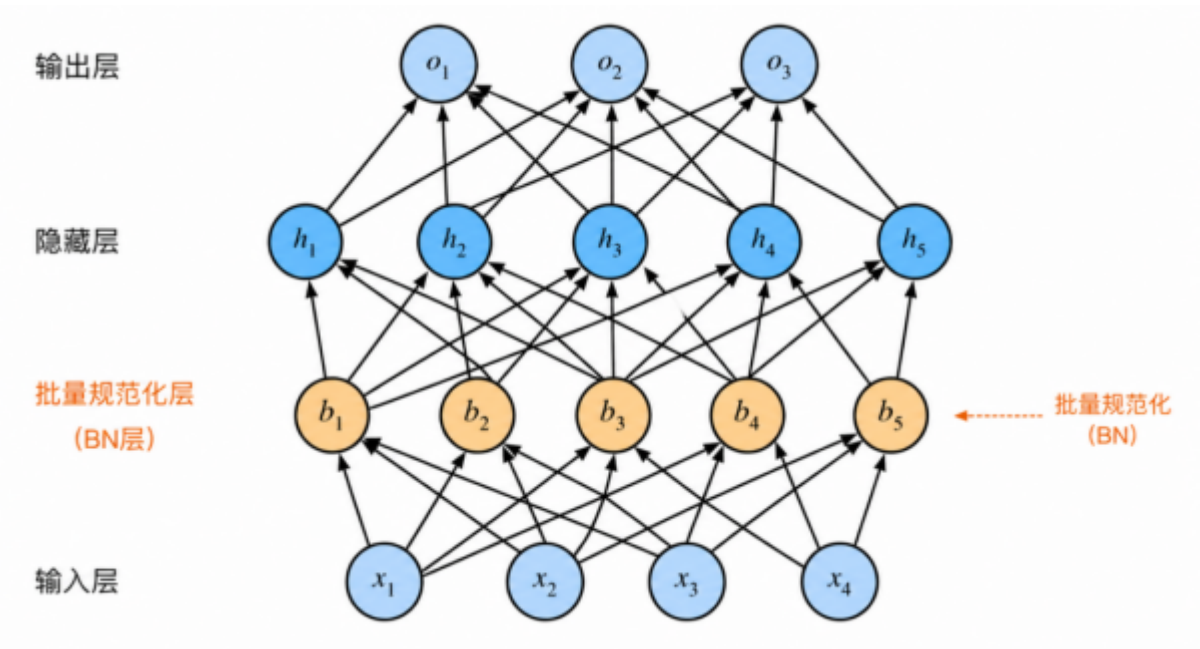
$$y = \gamma\hat{x} + \beta$$

其中 γ 为尺度参数，β 为偏移参数

总结起来，批量标准化先让数据稳定，再根据模型的需求，在训练的过程中学习尺度和偏移两个参数。

具体实现：

全连接层中：输入 -> 线性变换 -> 批量规范化层 -> 隐藏层（激活函数）-> 输出



如上图所示，输入层经过线性变换后，通过批量规范化层再进入隐藏层。（这种结构图，圆圈代表神经元，具体数字，变换是在箭头上发生的）

注意：全连接层中尺度，**偏移的参数量由隐藏层的神经元数量决定。**

为什么？因为批量规范化层的神经元个数和隐藏层的神经元个数是一样的。并且，批量规范化不是来一组数据（x_1 - x_4）就能处理的，而是来多组数据（一个 batch 里面有多个样本），对于线性变换后的每组数据的同位置数据进行规范化。

举例：（以上图为例）

输入：

1 个 batch = {样本一 $[x_{11}, x_{12}, x_{13}, x_{14}]$，样本二 $[x_{21}, x_{22}, x_{23}, x_{24}]$，样本三 $[x_{31}, x_{32}, x_{33}, x_{34}]$}

线性变换为：

样本 $1 \rightarrow [z_{11}, z_{12}, z_{13}, z_{14}, z_{15}]$

样本 $2 \rightarrow [z_{21}, z_{22}, z_{23}, z_{24}, z_{25}]$

样本 $3 \rightarrow [z_{31}, z_{32}, z_{33}, z_{34}, z_{35}]$

规范化：$[z_{11}, z_{21}, z_{31}]$ 算均值和方差，$[z_{12}, z_{22}, z_{32}]$ 算均值和方差.....

尺度/偏移系数个数：各 5 个

卷积层中：输入 -> 卷积 -> 批量规范化层 -> 隐藏层（激活函数）-> 输出

（批量规范化层的位置和全连接层中的很相似，只不过线性变换改成了卷积层）

和全连接层一样，只有一组数据（一个样本）是不行的，必须有一个 batch（多个样本）才能进行批量规范化。

输入（以图片为例）：batch_size（一个 batch 里面有多少样本）* 通道数（一个样本有多少张图）* 高 * 宽（一张图高多少，宽多少）

处理方法：先把这个 batch 中**所有样本的第一个通道**拿出来（得到 batch_size 张 高*宽 的数据表格），然后对这组数进行批量标准化。这个通道有自己的均值方差，两个训练参数。

所以，在卷积层中，**偏移的参数量由通道数决定。**

批量规范化在训练和测试时表现不同：（这一点和暂退法有共通之处）

训练时：每次训练，学习 γ、β，并计算当前 batch 的均值和方差。同时**维护两个量，来反映过去的训练中均值和方差的均值**，每次训练计算新的均值方差后更新这两个量。

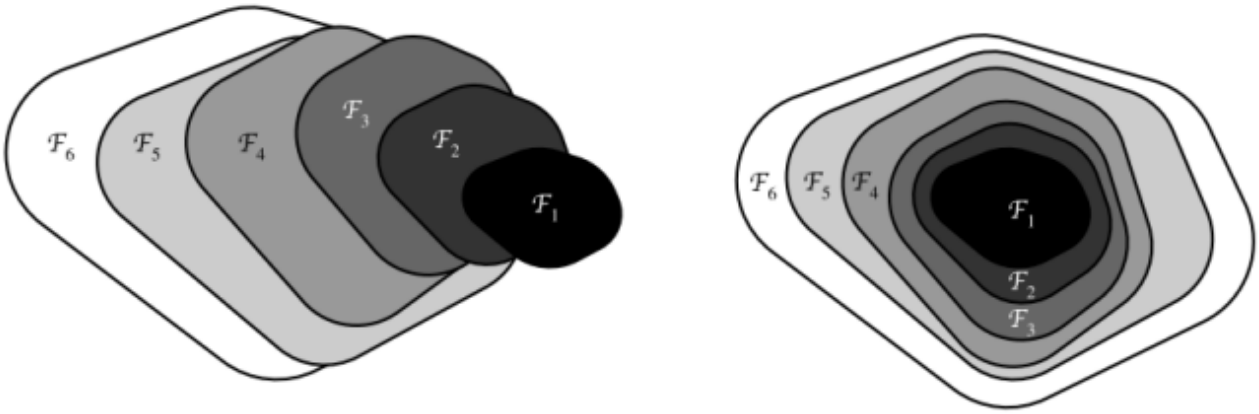
测试时：γ、β 就用学习好的，均值和方差用上面维护的那两个量来代替

（为什么这么做？测试时一般只会输入一组数据，样本量小，算出来的均值和方差不稳定）

残差网络（ResNet）

从一个常见的误区谈起：我们通常认为，在已有的网络上提高深度，必然能够获得比更浅的网络更好的训练效果。但实际上这是不对的。

以下图为例。F_1 到 F_6 的区域分别表示网络深度由浅变深时，能够得到的所有模型组成的集合。（为什么 F 大的区域更大？因为网络深度增加后，模型复杂度提高，能够表示的范围也就更大）



非嵌套函数类　　嵌套函数类

区别在于，左边的是非嵌套函数类，而右边的是嵌套函数类。假设我们寻找的最优解（对应图中某一点）处于 F_1 范围内。对于非嵌套函数类，我们发现当网络深度增加，有可能反而偏离/无法覆盖最优解，这显然是不好的。所以，我们希望能够变成右边的嵌套函数类，更深的模型能够覆盖原先模型的所有部分，也就是说**如果新增的复杂能力没用，它可以退回去使用原来简单模型的能力**。这就是残差网路（ResNet）想要做的事情。

下面是一个正常块和一个残差块的对比图

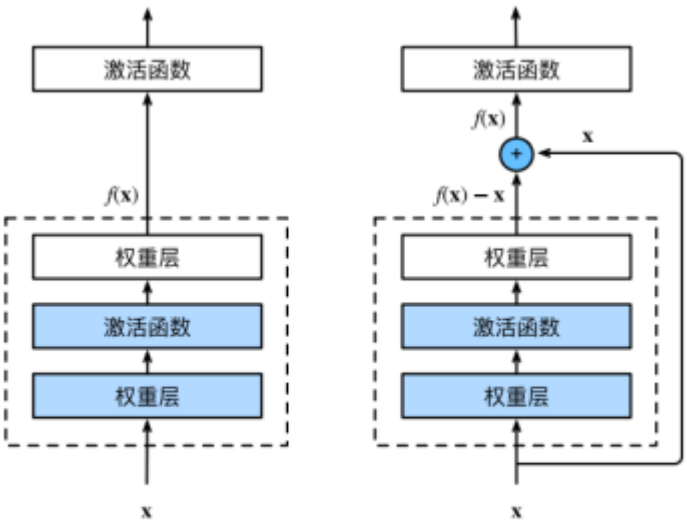


图7.6.2: 一个正常块（左图）和一个残差块（右图）。

注意 1：残差块**不是显式知道自己要学 $f(x) - x$，而是因为输出形式被设计成 $x + g(x)$，所以优化过程自然会让残差分支去学习"相对于 x 的修正量"**。当 x 已经足够好时，残差分支只需学成

0，就能保留原表示，从而让**加深后的网络至少不丢掉浅层网络的能力。**

为什么更容易成为嵌套函数类？因为接受一个输入 x，学成 0 要比学成 x 更简单

（直观理解：交白卷比 copy 一份原输入要更容易）

注意 2：上图中右边直接一个箭头连过去 $+x$ 实际上不是直接把 x 加到输出结果里。因为 x 经过虚线框内处理后和原来的 x 格式可能不相同

所以分两种情况：如果虚线框内是多层感知机，则旁路的 x 要先经过一个全连接层调整为同样规格的内容再去相加；如果虚线框内是卷积层，则旁路的 x 要经过 1×1 卷积层（同时调整步幅）处理为同样规格的内容再去相加。

还有一个好处：输入可以通过层间的残余连接更快地向前传播

理解：残差块有一条 shortcut，可以把输入 x 直接送到后面的相加处，不必完全经过中间所有层，因此有用信息更容易保留和传递，深层网络也更容易训练。

稠密连接网络（DenseNet）

既然后面的层可能需要前面层的信息，为什么不直接把所有前面层的特征都传给它？

构成：后面的层可以看到前面所有层的结果

Eg：普通 $x_0 \to x_1 \to x_2 \to x_3 \to x_4$

DenseNet：

$x_0 \to x_1$

$x_0, x_1 \to x_2$

$x_0, x_1, x_2 \to x_3$

$x_0, x_1, x_2, x_3 \to x_4$

与 ResNet 的区别：**ResNet 是相加，而 DenseNet 是拼接**

注意：DenseNet 不是整个网络从头到尾无限拼接，而是**分成若干个稠密块。**一个稠密块内部是密集连接的，保持空间尺寸不变，而通道数增加。

为了让参数量不爆炸，在稠密块之间加入**过渡层**（1×1 卷积和平均汇聚），减小通道数并降低空间尺寸。同时，对稠密块的每一层，都有**增长率** k，表示每一层额外贡献多少个新通道

（以上面的 $x_0, x_1, x_2, x_3 \to x_4$ 为例，x_0 假设有 64 通道，每一层的增长率 $k = 32$，则 x_4 就有 $64 + 4 \times 32 = 192$ 个通道)

所以 DenseNet 的整体结构大概是：

卷积层 → 稠密块 → 过渡层 → 稠密块 → 过渡层 → 稠密块 → 分类层

好处：特征复用更充分、梯度传播更容易、参数使用更高效

为什么我们在过渡层使用平均汇聚层而不是最大汇聚层？

因为 DenseNet 的过渡层主要目的不是"找最强特征"，而是平滑地压缩特征图尺寸，同时尽量保留已经积累下来的特征信息。